Purchase Order Management Process

From DRAFT to STOCK ENTRY

Process Description Document

Document Metadata

Field	Value
Document Version	1.0
Last Updated	February 4, 2026
Process Owner	Inventory Manager
Process ID	PROC-PO-001
Status	Active

---

1. Process Overview

Process Name

Purchase Order Management - Complete Lifecycle

Purpose & Business Value

This process enables the complete lifecycle management of purchase orders from initial draft creation by employees through admin approval, market execution, and final stock entry. It ensures proper

authorization controls, accurate inventory tracking, prevents duplicate stock entries, and maintains full audit compliance throughout the procurement workflow.

Success Metrics

- ✅ Purchase orders processed within 24 hours of submission
- ✅ Zero duplicate stock entries (enforced by system locks)
- ✅ 100% audit trail for all PO state transitions
- ✅ Variance tracking between confirmed and actual quantities

---

2. Stakeholders & Actors

Role	Responsibility	Permissions	Key Actions
EMPLOYEE	Create and manage purchase order drafts	Create, Edit DRAFT, Submit, Record found quantities	Draft PO, Submit for approval, Execute at market
ADMIN	Review, approve quantities, and authorize stock entry	Approve, Reject, Enter stock (final authorization)	Confirm quantities, Enter stock, Cancel PO

---

3. Purchase Order Status Flow

The purchase order progresses through **seven distinct states**, with strict validation rules at each transition:

Status Definitions

#	Status	Description	Who Can Act
1	**DRAFT**	Employee is building the order, can edit freely	EMPLOYEE (creator only)
2	**SUBMITTED**	Sent for admin approval, locked for editing	ADMIN (for approval/rejection)
3	**CONFIRMED**	Admin approved quantities, ready for market execution	EMPLOYEE (to execute at market)
4	**EXECUTED_AT_MARKET**	Employee recorded what was actually found at market	ADMIN (to approve stock entry)
5	**REJECTED_FOR_STOCK** ⚠️	Admin rejected stock entry (quantities invalid/mismatch)	EMPLOYEE (to correct) or ADMIN (to cancel)
6	**STOCK_ENTERED** ✅	Stock successfully added to inventory (FINAL STATE)	Read-only, no further actions
7	**CANCELLED** 🚫	Purchase order cancelled (TERMINAL STATE)	Read-only, no further actions

Status Flow Diagram

```
[DRAFT]
  ↓ (Employee submits)
[SUBMITTED]
  ↓ (Admin approves) ↺ (Admin rejects → back to DRAFT)
[CONFIRMED]
  ↓ (Employee executes at market)
```

```
[EXECUTED_AT_MARKET]
  ↓ (Admin reviews) ↺ (Admin rejects → REJECTED_FOR_STOCK)
[STOCK_ENTERED] ✅ FINAL
```

---

4. Detailed Process Flow

Phase 1: Order Creation (DRAFT)

Step	Actor	Action	System Response
1	EMPLOYEE	Clicks "Create Purchase Order"	Opens new PO form with auto-generated order number (PO-YYYY-NNNN)
2	EMPLOYEE	Searches and selects products to order	Displays product list with current stock levels and buying prices
3	EMPLOYEE	Enters desired quantity for each product	Calculates unit cost and total cost (read-only, from product.buyingPrice)
4	EMPLOYEE	Adds optional notes per item or for entire PO	Saves notes to purchaseOrderItems.notes or purchaseOrders.notes
5	EMPLOYEE	Clicks "Save as Draft"	Creates purchaseOrders record with status = DRAFT, saves all items
6	System	Validates data integrity	Ensures: products exist, quantities > 0, no duplicate products in same PO
7	System	Confirms save	Displays success message with PO number, allows further editing

Business Rules - DRAFT Phase

- ✅ Employee can only edit their own DRAFT purchase orders
- ✅ A product can only appear once per purchase order (unique constraint)
- ✅ Desired quantities must be positive integers (greater than 0)
- ✅ Unit cost is READ-ONLY, automatically pulled from product.buyingPrice
- ✅ Total cost = desiredQuantity × unitCost (calculated automatically)
- ✅ Employee can delete items or the entire draft before submission

---

Phase 2: Submission for Approval (SUBMITTED)

Step	Actor	Action	System Response
8	EMPLOYEE	Reviews draft PO and clicks "Submit for Approval"	Validates PO has at least one item with quantity > 0
9	System	Changes status to SUBMITTED	Sets submittedAt timestamp, locks PO from employee editing
10	System	Notifies admin users	Sends notification/email: "New PO pending approval: [PO-NUMBER]"
11	EMPLOYEE	Can view PO but cannot edit	Shows read-only view with "Awaiting Admin Approval" status badge

Business Rules - SUBMITTED Phase

- 🔒 SUBMITTED status locks the PO - employee cannot edit or delete
- 🔒 Only ADMIN users can view and act on SUBMITTED purchase orders

- 🔒 Employee can view status but must wait for admin decision
- 📝 System tracks submittedAt timestamp for audit purposes

Phase 3: Admin Approval (CONFIRMED)

Step	Actor	Action	System Response
12	ADMIN	Opens pending PO from admin dashboard	Displays PO details: items, desired quantities, costs
13	ADMIN	Reviews each item and adjusts confirmed quantities	Allows admin to approve exact, reduce, or increase quantities
14	ADMIN	Sets confirmedQuantity for each line item	Saves confirmedQuantity to purchaseOrderItems (can differ from desiredQuantity)
15	ADMIN	Clicks "Approve Purchase Order"	Changes status to CONFIRMED, sets confirmedAt timestamp
16	System	Validates all items have confirmedQuantity set	Rejects if any item missing confirmation, prompts admin to complete
17	System	Notifies employee that PO is approved	Sends notification: "PO [NUMBER] confirmed, proceed to market"

Alternative Flow - Admin Rejects PO

Step	Actor	Action	System Response
15a	ADMIN	Clicks "Reject" and provides rejection reason	Changes status back to DRAFT, notifies employee with rejection reason
16a	EMPLOYEE	Edits PO based on feedback, resubmits	Process returns to Phase 2 (SUBMITTED)

Business Rules - CONFIRMED Phase

- ✅ Admin must set confirmedQuantity for ALL items before approving
- ✅ confirmedQuantity can be equal to, less than, or greater than desiredQuantity
- ✅ confirmedQuantity must be a positive integer (greater than 0)
- ✅ confirmedQuantity of 0 effectively removes item from order
- 📝 System tracks confirmedAt timestamp and admin who confirmed
- 🔒 Once CONFIRMED, admin cannot edit quantities (would require moving back to DRAFT)

Phase 4: Market Execution (EXECUTED_AT_MARKET)

Step	Actor	Action	System Response
18	EMPLOYEE	Takes confirmed PO to market/supplier	Views mobile-friendly PO showing confirmed quantities to purchase
19	EMPLOYEE	Attempts to purchase items at market	May find exact quantity, less, or more depending on availability
20	EMPLOYEE	Records actualFoundQuantity for each item	Opens market execution form, enters what was actually obtained

Step	Actor	Action	System Response
21	EMPLOYEE	Enters actualFoundQuantity (can be 0 if item unavailable)	Saves to purchaseOrderItems.actualFoundQuantity field
22	EMPLOYEE	Adds notes explaining variances if needed	Records reasons: item out of stock, supplier changed, price difference, etc.
23	EMPLOYEE	Clicks "Complete Market Execution"	Validates all items have actualFoundQuantity recorded
24	System	Changes status to EXECUTED_AT_MARKET	Sets executedAt timestamp, calculates variance report
25	System	Generates variance report	Shows: Confirmed vs Actual for each item, flags large discrepancies
26	System	Notifies admin that PO is ready for stock entry review	Alert: "PO [NUMBER] executed, awaiting stock entry approval"

Business Rules - EXECUTED_AT_MARKET Phase

- ✅ actualFoundQuantity must be set for ALL items (can be 0 if not found)
- ✅ actualFoundQuantity can differ from confirmedQuantity (real-world scenario)
- 📊 System calculates variance: `variance = actualFoundQuantity - confirmedQuantity`
- ⚠️ Large variances (e.g., more than 20% difference) should trigger admin review flag
- 📝 Employee should document reason for variances in notes field
- 🔒 Once EXECUTED_AT_MARKET, only admin can proceed or reject

---

Phase 5: Stock Entry Approval & Final Stock Addition (STOCK_ENTERED)

Step	Actor	Action	System Response
27	ADMIN	Reviews executed PO and variance report	Displays comparison: Desired vs Confirmed vs Actual Found
28	ADMIN	Verifies quantities are reasonable and documented	Checks for excessive variances, missing justifications
29	ADMIN	Clicks "Enter Stock" to approve final entry	**CRITICAL**: System checks if stockEnteredAt is NULL (prevents double entry)
30	System	**VALIDATES HARD LOCK**: stockEnteredAt must be NULL	If stockEnteredAt already set: **ERROR** - Stock already entered for this PO
31	System	Begins atomic transaction (all or nothing)	Starts database transaction to ensure data integrity
32	System	For each item: Updates stocks.quantity += actualFoundQuantity	Adds quantities to existing stock records for each product
33	System	Creates stockActions records (audit trail)	Logs: productId, actionType=STOCK_IN, quantity=actualFoundQuantity, doneBy=adminId
34	System	**Sets stockEnteredAt = NOW() (PERMANENT LOCK)**	Once set, this PO can NEVER have stock entered again
35	System	Changes status to STOCK_ENTERED (final state)	PO is now complete and read-only
36	System	Commits transaction	All stock updates, audit logs, and status change saved together

Step	Actor	Action	System Response
37	System	Displays success confirmation	Shows: Stock successfully added, PO completed, audit trail created

Alternative Flow - Admin Rejects Stock Entry

Step	Actor	Action	System Response
29a	ADMIN	Clicks "Reject for Stock Entry" with reason	Changes status to REJECTED_FOR_STOCK, notifies employee
30a	EMPLOYEE	Corrects actualFoundQuantity based on admin feedback	Re-submits for admin review, returns to Phase 5

CRITICAL Business Rules - STOCK_ENTERED Phase

🔒 **HARD LOCK ENFORCEMENT (Prevents Double Stock Entry):**

- ✅ `stockEnteredAt` timestamp acts as a ONE-TIME LOCK
- ✅ System MUST check `stockEnteredAt IS NULL` before allowing stock entry
- ⛔ If `stockEnteredAt` is NOT NULL: REJECT with error message
- ✅ Once `stockEnteredAt` is set, it can NEVER be changed or cleared
- ✅ This prevents accidental or malicious duplicate stock additions

Additional Rules:

- ✅ All stock updates must happen in a single atomic transaction
- ✅ stockActions records must be created for EVERY product with actualFoundQuantity > 0
- ✅ Audit trail must include: admin who approved, timestamp, quantities added
- 🔒 Status change to STOCK_ENTERED is irreversible (terminal state)

- 📖 After STOCK_ENTERED, PO becomes read-only for historical reference

---

5. Data Validation & Business Rules Summary

Validation at Each Stage

Stage	Required Validations	Error Handling
DRAFT Creation	• At least 1 item • All products exist • No duplicate products • desiredQuantity > 0 • unitCost matches product.buyingPrice	Display inline errors, prevent save until fixed
SUBMITTED	• User is creator • Status is DRAFT • PO has items	Reject if not creator or wrong status, show error message
CONFIRMED	• User is ADMIN • Status is SUBMITTED • All items have confirmedQuantity > 0	Reject if not admin, highlight items missing confirmation
EXECUTED_AT_MARKET	• Status is CONFIRMED • All items have actualFoundQuantity (≥ 0) • Variances documented if > 20%	Reject if wrong status, highlight missing quantities, warn on large variances

Stage	Required Validations	Error Handling
STOCK_ENTERED	• User is ADMIN • Status is EXECUTED_AT_MARKET • **stockEnteredAt IS NULL** • All products exist in stocks table	**CRITICAL**: If stockEnteredAt NOT NULL, show ERROR: "Stock already entered for this PO". Rollback transaction on any failure.

6. Security & Permissions

Action	EMPLOYEE	ADMIN
Create Draft PO	✓ Yes	✓ Yes
Edit Own Draft	✓ Only own	✓ Any
Submit for Approval	✓ Only own	✗ No need
Approve/Confirm Quantities	✗ No	✓ Yes
Record Market Execution	✓ Yes	✓ Yes
Enter Stock (Final)	✗ No	✓ Yes (ONLY)
Cancel PO	✓ Only own DRAFT	✓ Any status

7. Audit & Compliance

Required Audit Logs

Every state transition and critical action must be logged in the `activityLogs` table with:

- **userId**: Who performed the action
- **action**: Descriptive action name (e.g., 'PO_SUBMITTED', 'PO_STOCK_ENTERED')
- **entityType**: 'PURCHASE_ORDER'
- **entityId**: Purchase order ID
- **details**: JSON containing relevant data (quantities, status changes, etc.)
- **doneAt**: Timestamp when action occurred

Critical Audit Events

1. ✅ PO created (DRAFT)
2. ✅ PO submitted for approval (DRAFT → SUBMITTED)
3. ✅ PO approved with confirmed quantities (SUBMITTED → CONFIRMED)
4. ✅ PO rejected by admin (SUBMITTED → DRAFT)
5. ✅ Market execution recorded (CONFIRMED → EXECUTED_AT_MARKET)
6. ✅ Stock entry attempted (log even if fails due to stockEnteredAt lock)
7. ✅ Stock successfully entered (EXECUTED_AT_MARKET → STOCK_ENTERED)
8. ✅ Stock entry rejected (EXECUTED_AT_MARKET → REJECTED_FOR_STOCK)
9. ✅ PO cancelled (any status → CANCELLED)

Stock Action Audit Trail

When stock is entered (Phase 5, Step 33), the system must create `stockActions` records:

Field	Value
productId	Product being stocked
actionType	STOCK_IN
quantity	actualFoundQuantity from purchase order item
buyingPrice	unitCost from purchase order item
sellingPrice	NULL (not set during purchase)
supplier	Optional: supplier name if recorded
reason	"Purchase Order: [PO_NUMBER]" with optional notes
doneBy	Admin user ID who entered stock
doneAt	Current timestamp

---

8. Error Scenarios & Handling

Scenario A: Double Stock Entry Attempt (CRITICAL)

| **Trigger** | Admin attempts to enter stock for PO that already has stockEnteredAt set | | **System Check** | `SELECT stockEnteredAt FROM purchaseOrders WHERE id = [PO_ID]` | | **Condition** | `IF stockEnteredAt IS NOT NULL` | | **Response** | **ERROR**: "Stock has already been entered for this purchase order on [DATE]. This operation cannot be performed again." | | **Logging** | Log attempted double entry in activityLogs with userId and PO ID | | **Prevention** | UI should disable "Enter Stock" button if stockEnteredAt is not NULL |

Scenario B: Transaction Rollback on Stock Entry Failure

| **Trigger** | Database error, network failure, or validation failure during stock entry | | **Examples** | Product not found, database deadlock, insufficient permissions | | **Response** | ROLLBACK entire transaction - no partial stock updates allowed | | **User Message** | "Failed to enter stock. No changes have been made. Please try again or contact support." | | **Technical Log** | Log full error details, stack trace, affected PO and products | | **Status** | PO remains in EXECUTED_AT_MARKET status, stockEnteredAt remains NULL |

Scenario C: Employee Editing Submitted PO

| **Trigger** | Employee attempts to edit PO with status = SUBMITTED or later | | **Response** | "This purchase order has been submitted and is locked for editing. Contact an administrator if changes are needed." | | **UI Behavior** | All form fields should be read-only, edit/delete buttons hidden | | **Exception** | If admin sends back to DRAFT via rejection, employee can edit again |

Scenario D: Large Variance Between Confirmed and Actual Quantities

| **Trigger** | actualFoundQuantity differs from confirmedQuantity by more than 20% | | **Detection** | System calculates: `ABS((actual - confirmed) / confirmed) > 0.20` | | **Warning** | Display warning badge: "⚠️ Large variance detected - requires justification" | | **Required Action** | Employee must provide detailed notes explaining variance reason | | **Admin Review** | Admin can accept with justification or reject for correction | | **Examples** | Confirmed: 100, Actual: 75 → 25% variance
Confirmed: 50, Actual: 65 → 30% variance |

---

9. Expected Outcomes & Success Criteria

Functional Success

- ✅ Purchase order progresses through all states correctly

- ✅ Stock quantities update accurately based on actualFoundQuantity
- ✅ Double stock entry is prevented by stockEnteredAt lock
- ✅ All state transitions are logged in activityLogs
- ✅ stockActions audit trail is created for all stock additions

User Experience Success

- ✅ Process completes within 2 business days from DRAFT to STOCK_ENTERED
- ✅ Clear status indicators at every stage
- ✅ Helpful error messages with actionable guidance
- ✅ Mobile-friendly interface for market execution phase

Business Success

- ✅ 100% audit compliance for all stock movements
- ✅ Zero duplicate stock entries (enforced by system)
- ✅ Variance tracking improves purchasing accuracy over time
- ✅ Clear separation of duties between employees and admins

---

10. Data Flow Summary

Key Database Tables Involved

Table	Purpose	Updated When
purchaseOrders	Header record for each PO	Created in DRAFT, status updated at each phase, stockEnteredAt set once

Table	Purpose	Updated When
purchaseOrderItems	Line items for each product	desiredQuantity in DRAFT, confirmedQuantity in CONFIRMED, actualFoundQuantity in EXECUTED_AT_MARKET
stocks	Current inventory quantities	Updated in STOCK_ENTERED phase (`quantity += actualFoundQuantity`)
stockActions	Audit trail for stock movements	Created in STOCK_ENTERED phase for each product
activityLogs	Audit trail for all user actions	At every status change and critical action

11. Visual Process Flow

```
┌──────────┐
│ DRAFT    │ ← Employee creates and edits
└──────────┘
    │ Submit
    ↓
┌──────────────┐
│ SUBMITTED    │ ← Admin reviews
└──────────────┘
    │   └──────→ Reject → Back to DRAFT
    │ Approve
    ↓
┌──────────────┐
│ CONFIRMED    │ ← Employee executes at market
└──────────────┘
    │
```

```
  ↓
┌──────────────────────┐
│EXECUTED_AT_MARKET│ ← Admin reviews execution
└──────────┬──────────┘
   │   └──────→ Reject → REJECTED_FOR_STOCK
   │ Approve
  ↓
┌──────────────────┐
│ STOCK_ENTERED │ ✅ FINAL STATE (Read-only)
└──────────────────┘
```

---

12. Database Schema Reference

purchaseOrders Table

```typescript
```

```
{
  id: uuid (PK),
  orderNumber: varchar(50) UNIQUE,
  status: purchaseOrderStatusEnum,
  createdBy: uuid → users.id,

  // Timestamps
  submittedAt: timestamp,
  confirmedAt: timestamp,
  executedAt: timestamp,
  stockEnteredAt: timestamp, // 🔒 HARD LOCK (one-time only)

  notes: text,
  createdAt: timestamp,
  updatedAt: timestamp
}
```

purchaseOrderItems Table

```typescript
```

```
{
  id: uuid (PK),
  purchaseOrderId: uuid → purchaseOrders.id,
  productId: uuid → products.id,

  // Three-stage quantities
  desiredQuantity: integer,      // Step 1: Employee intent
  confirmedQuantity: integer,    // Step 2: Admin approval
  actualFoundQuantity: integer,  // Step 3: Market reality

  // Price snapshot (READ-ONLY)
  unitCost: decimal(10,2),
  totalCost: decimal(12,2),

  notes: text,
  createdAt: timestamp,
  updatedAt: timestamp,

  UNIQUE(purchaseOrderId, productId) // One product per PO
}
```

---

13. Notes & Assumptions

Assumptions

- ✅ Products are pre-configured in the products table before PO creation
- ✅ Users have stable internet connection for real-time updates
- ✅ Admin users are available within 24 hours to review submitted POs
- ✅ Stock table has existing records for all products (quantity can be 0)

Known Limitations

- ⚠️ No offline mode - requires active database connection
- ⚠️ No bulk PO creation (one PO at a time)
- ⚠️ Cannot split actualFoundQuantity across multiple stock locations
- ⚠️ No automated notifications (must be implemented separately)

Future Enhancements (Out of Scope)

- 🔄 Multi-level approval workflow (manager → senior admin → finance)
- 🔄 Automated supplier integration for real-time pricing
- 🔄 Mobile app with barcode scanning for market execution
- 🔄 Predictive ordering based on historical consumption patterns
- 🔄 Supplier performance tracking and ratings

---

14. Document Change History

Version	Date	Author	Changes
1.0	Feb 4, 2026	Process Team	Initial process documentation created

---

15. Stakeholder Sign-Off

Stakeholder	Role	Status	Date
[Name]	Product Owner	🔄 Pending	-

Stakeholder	Role	Status	Date
[Name]	IT Manager	🔄 Pending	-
[Name]	Inventory Manager	🔄 Pending	-
[Name]	Development Lead	🔄 Pending	-

✅ Ready for Development Checklist

- [ ] All required fields completed
- [ ] Business rules clearly defined
- [ ] Stakeholders have signed off
- [ ] Technical feasibility confirmed
- [ ] Security review completed
- [ ] Database schema validated
- [ ] Audit requirements documented

— End of Document —

This document serves as the contract between business stakeholders and the development team. No development work should begin until all stakeholders have approved this process.